

EXPERIMENT NO. 04

Aim : Create a transformer from scratch using the Pytorch librar.

Theory:

Transformer Model: The Transformer is a powerful neural network architecture that has been shown to achieve state-of-the-art performance on a wide range of natural language processing tasks, including language modeling, machine translation, and sentiment analysis.

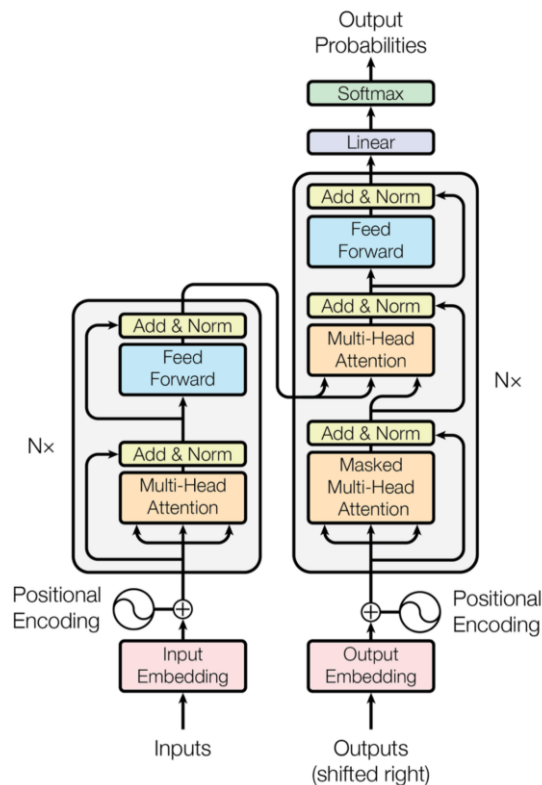


Figure 1: The Transformer - model architecture.

Figure :. The Transformer Network

Architecture

The Transformer architecture consists of a set of encoders and a decoders, In the paper they used 6 of each.

The encoder processes the input sequence of tokens, while the decoder generates the output sequence.

Here's a brief overview of each component:

Input Embedding

The input sequence of tokens is first embedded into a continuous vector space using an embedding layer.

This layer maps each token to a fixed-length vector that captures its semantic meaning. The embedding layer is trained jointly with the rest of the model.

Positional Encoding

Since the Transformer does not use recurrent or LSTM layers, it needs a way to incorporate positional information about the tokens. This is done using a positional encoding layer, which adds a set of sinusoidal (sin and cos) functions to the input embeddings. The frequencies and phases of these functions encode the position of each token in the input sequence.

Encoder Layers

The encoder consists of multiple identical layers, each of which applies a set of operations to the input sequence in parallel. The core operation of each encoder layer is multi-head self-attention (they used 8 heads in the paper), which allows the model to attend to different parts of the input sequence at different levels of granularity.

The outputs of the self-attention layer are passed through a feedforward neural network, and the resulting representations are combined with the original input sequence using residual connections and layer normalization.

Decoder Layers

The decoder also consists of multiple identical layers, each of which applies a similar set of operations to the output sequence. In addition to the self-attention and feedforward layers, each decoder layer also includes a multi-head attention layer that attends to the encoder output. This allows the decoder to align the input and output sequences and generate more accurate translations.

Output Layer

The final layer of the decoder is a softmax layer that produces a probability distribution over the output vocabulary. During training, the model is trained to maximize the likelihood of the correct output sequence. During inference, the output sequence is generated by sampling from the predicted probability distribution.

Merging it all together:

```
class Transformer(nn.Module):
    def __init__(self, src_vocab_size, tgt_vocab_size, d_model, num_heads, num_layers, d_ff,
max_seq_length, dropout):
        super(Transformer, self).__init__()
        self.encoder_embedding = nn.Embedding(src_vocab_size, d_model)
        self.decoder_embedding = nn.Embedding(tgt_vocab_size, d_model)
        self.positional_encoding = PositionalEncoding(d_model, max_seq_length)

        self.encoder_layers = nn.ModuleList([EncoderLayer(d_model, num_heads, d_ff, dropout)
for _ in range(num_layers)])
        self.decoder_layers = nn.ModuleList([DecoderLayer(d_model, num_heads, d_ff, dropout)
for _ in range(num_layers)])

        self.fc = nn.Linear(d_model, tgt_vocab_size)
        self.dropout = nn.Dropout(dropout)

    def generate_mask(self, src, tgt):
        src_mask = (src != 0).unsqueeze(1).unsqueeze(2)
        tgt_mask = (tgt != 0).unsqueeze(1).unsqueeze(3)
        seq_length = tgt.size(1)
        nopeak_mask = (1 - torch.triu(torch.ones(1, seq_length, seq_length), diagonal=1)).bool()
        tgt_mask = tgt_mask & nopeak_mask
        return src_mask, tgt_mask

    def forward(self, src, tgt):
        src_mask, tgt_mask = self.generate_mask(src, tgt)
        src_embedded = self.dropout(self.positional_encoding(self.encoder_embedding(src)))
        tgt_embedded = self.dropout(self.positional_encoding(self.decoder_embedding(tgt)))

        enc_output = src_embedded
        for enc_layer in self.encoder_layers:
            enc_output = enc_layer(enc_output, src_mask)

        dec_output = tgt_embedded
        for dec_layer in self.decoder_layers:
            dec_output = dec_layer(dec_output, enc_output, src_mask, tgt_mask)
```

```
output = self.fc(dec_output)
return output
```

The Transformer class combines the previously defined modules to create a complete Transformer model. During initialization, the Transformer module sets up input parameters and initializes various components, including embedding layers for source and target sequences, a PositionalEncoding module, EncoderLayer and DecoderLayer modules to create stacked layers, a linear layer for projecting decoder output, and a dropout layer.

The generate_mask method creates binary masks for source and target sequences to ignore padding tokens and prevent the decoder from attending to future tokens. The forward method computes the Transformer model's output through the following steps:

1. Generate source and target masks using the generate_mask method.
2. Compute source and target embeddings, and apply positional encoding and dropout.
3. Process the source sequence through encoder layers, updating the enc_output tensor.
4. Process the target sequence through decoder layers, using enc_output and masks, and updating the dec_output tensor.

5. Apply the linear projection layer to the decoder output, obtaining output logits.

These steps enable the Transformer model to process input sequences and generate output sequences based on the combined functionality of its components.

Preparing Sample Data

In this example, we will create a toy dataset for demonstration purposes. In practice, you would use a larger dataset, preprocess the text, and create vocabulary mappings for source and target languages.

```
src_vocab_size = 5000
tgt_vocab_size = 5000
d_model = 512
num_heads = 8
num_layers = 6
d_ff = 2048
max_seq_length = 100
dropout = 0.1

transformer = Transformer(src_vocab_size, tgt_vocab_size, d_model, num_heads,
                          num_layers, d_ff, max_seq_length, dropout)

# Generate random sample data
src_data = torch.randint(1, src_vocab_size, (64, max_seq_length)) # (batch_size, seq_length)
tgt_data = torch.randint(1, tgt_vocab_size, (64, max_seq_length)) # (batch_size, seq_length)
```

Training the Model

Now we'll train the model using the sample data. In practice, you would use a larger dataset and split it into training and validation sets.

```
criterion = nn.CrossEntropyLoss(ignore_index=0)
optimizer = optim.Adam(transformer.parameters(), lr=0.0001, betas=(0.9, 0.98), eps=1e-9)

transformer.train()

for epoch in range(100):
    optimizer.zero_grad()
    output = transformer(src_data, tgt_data[:, :-1])
    loss = criterion(output.contiguous().view(-1, tgt_vocab_size), tgt_data[:,
1:].contiguous().view(-1))
    loss.backward()
    optimizer.step()
    print(f"Epoch: {epoch+1}, Loss: {loss.item()}")
```

We can use this way to build a simple Transformer from scratch in Pytorch. All Large Language Models use these Transformer encoder or decoder blocks for training. Hence understanding the network that started it all is extremely important. Hope this article helps all looking to deep dive into LLM's.